SNOWFLAKE PORTFOLIO PROJECT

LAYER: INFRASTRUCTURE

Snowflake: Healthcare Claims Database Setup

Infrastructure Layer -- Schema, Stage, Table, Load Pipeline

Snowflake | SQL | DDL | ETL | Healthcare | Cloud Warehouse | CSV Stage | COPY INTO

Project Summary

Establishes the full Snowflake infrastructure for the healthcare claims analytics platform. Creates the database schema, a 14-column typed claims table, and a CSV external stage with NULL handling and date parsing. Loads data via COPY INTO with ON_ERROR=CONTINUE, then verifies row counts. This infrastructure layer underpins all 14 DBT models in the healthcare analytics pipeline.

Thomas E. Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com | (540) 570-6094

Pea Ridge, AR 72751

1 PROJECT OVERVIEW

Platform:	Snowflake Cloud Data Warehouse
Database:	HEALTHCARE_DB
Schema:	HEALTHCARE_SCHEMA
Stage Name:	claims_stage
Table:	CLAIMS (14 columns, typed)
Load Method:	COPY INTO from CSV stage file
File Format:	CSV with header skip, quoted fields, date formatting
NULL Handling:	NULL_IF = (NULL, null, empty string)
Error Policy:	ON_ERROR = CONTINUE (skip bad rows)
Row Volume:	~50,000 healthcare claims time series records
Audience:	Data Engineers, DBT Developers, Analytics Engineers

2 BUSINESS CONTEXT

This script establishes the foundational data infrastructure for the entire healthcare claims analytics project. Without a properly configured Snowflake schema and typed table, the 14 downstream DBT models -- covering staging, rolling averages, regression, ARIMA, GARCH, Value at Risk, Sharpe Ratio, and the combined mart -- cannot execute.

- Schema isolation: HEALTHCARE_SCHEMA separates production analytics from sandbox demo schemas.
- Type safety: All 14 columns are explicitly typed (VARCHAR, DATE, FLOAT, INT) -- no VARIANT catch-alls in the final table.
- Stage-based loading: An external named stage decouples file location from table DDL, making reruns and file swaps trivial.
- ON_ERROR=CONTINUE: Allows partial loads without blocking the pipeline on malformed rows -- bad rows are skipped and logged.
- GRANT WRITE: Explicit privilege assignment ensures least-privilege access without defaulting to SYSADMIN for every operation.
- Verification SELECTs: COUNT(*) and LIMIT 10 queries confirm load success before DBT models run.
- DROP SCHEMA at end: Cleans up the demo.dbt_tfoster sandbox after development -- keeps Snowflake tidy.

3 COLUMN REFERENCE -- CLAIMS TABLE

The CLAIMS table contains 14 columns spanning entity identifiers, time series keys, financial metrics, and model-specific predictor values. All columns are nullable to accommodate partial claim submissions.

Column	Snowflake Type	Role / Description
entity_id	VARCHAR	Unique identifier for provider, payer, or patient entity
payer	VARCHAR	Insurance payer name (e.g., Medicare, Medicaid, Commerci
state	VARCHAR(2)	2-letter state code of claim origin
specialty	VARCHAR	Medical specialty of the submitting provider
period_date	DATE	Reporting period date -- primary time series key
metric_value	FLOAT	Primary claim metric (e.g., allowed amount, cost index)
claim_volume	INT	Count of claims filed in the reporting period
return_value	FLOAT	Financial return metric used in risk / Sharpe models
denial_rate	FLOAT	Claim denial rate as decimal (0.0 to 1.0)
days_to_payment	INT	Days from claim submission to payment settlement
is_fraud_flag	INT	Binary fraud indicator: 1=flagged, 0=clean
y_value	FLOAT	Dependent variable for regression / statistical models
x1_value	FLOAT	First independent predictor for multi-linear regression
x2_value	FLOAT	Second independent predictor for multi-linear regression

4 STAGE CONFIGURATION

The named stage uses Snowflake CSV file format parameters to ensure clean parsing of the healthcare CSV file:

- TYPE = 'CSV' -- treats the staged file as comma-separated values.
- FIELD_OPTIONALLY_ENCLOSED_BY = '"' -- handles quoted fields containing commas (e.g., specialty names).
- SKIP_HEADER = 1 -- skips the CSV header row on every COPY INTO execution.
- DATE_FORMAT = 'YYYY-MM-DD' -- parses period_date strings into Snowflake DATE type without casting.
- NULL_IF = ('NULL','null','') -- converts three common null representations to Snowflake NULL.

5 COMPLETE SNOWFLAKE SETUP SCRIPT

Full DDL and load script -- no masking applied. This is portfolio-safe infrastructure code with no sensitive credentials.

Script -- Steps 1-2: Schema and Stage Creation

```
-- ============================================================
--  Snowflake Healthcare Claims Database Infrastructure Setup
--  Author : Thomas E. Foster
--  Purpose: Create schema, stage, typed table, and load data
-- ============================================================

-- Step 1: Create Schema
CREATE OR REPLACE SCHEMA HEALTHCARE_DB.HEALTHCARE_SCHEMA;

-- Step 2a: Placeholder table (VARIANT) before final typed DDL
CREATE OR REPLACE TABLE HEALTHCARE_DB.HEALTHCARE_SCHEMA.CLAIMS (data VARIANT);

-- Step 2b: Named External Stage with CSV file format
CREATE OR REPLACE STAGE HEALTHCARE_DB.HEALTHCARE_SCHEMA.claims_stage
  FILE_FORMAT = (
    TYPE                         = 'CSV'
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER                  = 1
    DATE_FORMAT                  = 'YYYY-MM-DD'
    NULL_IF                      = ('NULL', 'null', '')
  );
```

Script -- Step 3: Final Typed Table DDL

```
-- Step 3: Final typed CLAIMS table (14 columns)
CREATE OR REPLACE TABLE HEALTHCARE_DB.HEALTHCARE_SCHEMA.CLAIMS (
  entity_id       VARCHAR,
  payer           VARCHAR,
  state           VARCHAR(2),
  specialty       VARCHAR,
  period_date     DATE,
  metric_value    FLOAT,
  claim_volume    INT,
  return_value    FLOAT,
  denial_rate     FLOAT,
  days_to_payment INT,
  is_fraud_flag   INT,
  y_value         FLOAT,
  x1_value        FLOAT,
  x2_value        FLOAT
);
```

5 COMPLETE SNOWFLAKE SETUP SCRIPT (CONTINUED)

Script -- Steps 4-8: Role Check, Grant, COPY INTO, Verify, Cleanup

```
-- Step 4: Check active role
SELECT CURRENT_ROLE();

-- Step 5: Grant WRITE on stage to ACCOUNTADMIN (if not already on privileged role)
GRANT WRITE ON STAGE HEALTHCARE_DB.HEALTHCARE_SCHEMA.claims_stage
  TO ROLE ACCOUNTADMIN;

-- Step 6: Load CSV from stage into CLAIMS table
COPY INTO HEALTHCARE_DB.HEALTHCARE_SCHEMA.CLAIMS
  FROM @HEALTHCARE_DB.HEALTHCARE_SCHEMA.claims_stage/healthcare_claims_timeseries.csv
  FILE_FORMAT = (
    TYPE                         = 'CSV'
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER                  = 1
    DATE_FORMAT                  = 'YYYY-MM-DD'
    NULL_IF                      = ('NULL', 'null', '')
  )
  ON_ERROR = 'CONTINUE';

-- Step 7: Verify load
SELECT COUNT(*) FROM HEALTHCARE_DB.HEALTHCARE_SCHEMA.CLAIMS;
SELECT * FROM HEALTHCARE_DB.HEALTHCARE_SCHEMA.CLAIMS LIMIT 10;

-- Step 8: Cleanup demo sandbox schema (post-development)
DROP SCHEMA DEMO.DBT_TFOSTER;
```

6 TECHNICAL EXPLANATION

Step-by-step explanation of each SQL block and the design decisions behind them:

- CREATE OR REPLACE SCHEMA: Idempotent -- safe to re-run. Creates HEALTHCARE_SCHEMA under HEALTHCARE_DB, isolating all tables from other schemas.
- VARIANT placeholder table: A VARIANT table allows quick inspection of raw JSON or CSV data before committing to a typed schema. Replaced immediately by the typed DDL.
- Named stage (claims_stage): Decouples the file location from the table DDL. The stage stores file format defaults, so COPY INTO inherits them without repeating parameters.
- Typed CLAIMS table: 14 explicitly typed columns eliminate implicit casting at query time. DATE, FLOAT, INT types are optimized for Snowflake columnar storage and DBT downstream models.
- GRANT WRITE: Explicit privilege ensures the role running COPY INTO has write access. Best practice to avoid running everything as ACCOUNTADMIN.
- COPY INTO with ON_ERROR=CONTINUE: Snowflake skips malformed rows instead of failing the entire load. Rejected rows are logged in COPY_HISTORY for auditing.
- Verification SELECTs: COUNT(*) confirms the expected row count loaded; LIMIT 10 does a quick sanity check on column types and values.
- DROP SCHEMA DEMO.DBT_TFOSTER: Drops the development sandbox schema after the production schema is validated -- keeps Snowflake credits and storage lean.

7 PIPELINE ARCHITECTURE -- WHERE THIS FITS

This Snowflake setup script is Layer 0 of a 4-layer analytics pipeline. Every subsequent DBT model references the CLAIMS table created here.

Layer 0 -- Infrastructure	This script: Schema + Stage + Table + COPY INTO load
Layer 1 -- Staging (DBT)	stg_healthcare_claims.sql -- type casting, NULL cleanup, column aliasing
Layer 2 -- Analytics (DBT)	12 analytical models: rolling averages, regression, ARIMA, GARCH, VaR, Sharpe
Layer 3 -- Mart (DBT)	mart_combined.sql -- wide table joining all Layer 2 models for BI consumption
Layer 4 -- Visualization	Tableau dashboards connecting directly to HEALTHCARE_SCHEMA via Snowflake connector

8 EXECUTION PREREQUISITES

- Snowflake account with HEALTHCARE_DB database already created (CREATE DATABASE HEALTHCARE_DB if not exists).
- ACCOUNTADMIN or SYSADMIN role assigned to the executing session.
- CSV file (healthcare_claims_timeseries.csv) uploaded to the named stage via: PUT file:///path/to/file @HEALTHCARE_DB.HEALTHCARE_SCHEMA.claims_stage;
- Snowflake virtual warehouse must be active (RESUME if suspended) -- COPY INTO requires compute.
- DBT profile (profiles.yml) must reference HEALTHCARE_DB.HEALTHCARE_SCHEMA as the target schema.

9 SKILLS DEMONSTRATED

- Snowflake DDL: CREATE OR REPLACE SCHEMA, TABLE, STAGE with typed columns
- CSV file format configuration: header skip, quoting, date formatting, NULL normalization
- Stage-based ELT loading: COPY INTO from named stage with ON_ERROR fault tolerance
- Privilege management: GRANT WRITE on stage for least-privilege access control
- Data verification: COUNT(*) and LIMIT checks as part of the load pipeline
- Environment hygiene: DROP SCHEMA for sandbox cleanup post-development
- End-to-end pipeline thinking: Infrastructure designed specifically for DBT + Tableau consumption